

AI-Assisted Software Development with AI Blueprint

A practical operating manual for Codex / Claude Code

Build deliberately. Review continuously. Keep the human in control.

Research date: 26 September 2026

1. Purpose of This Guide

This guide turns the supplied YouTube video, AI Blueprint GitHub repository, and official AI Blueprint documentation into a practical workflow you can follow while building your own software. The objective is not to make an AI agent generate an entire application from one huge prompt. The objective is to use AI as a disciplined implementation partner while you remain the product owner, architect, reviewer and release owner.

Core idea: Think in scoped work items rather than giant prompts: plan → specify → approve → implement → prove → review → complete.

Most important habit: Never allow an unclear requirement to become a large architectural decision silently.

Source authority: For current command names and behavior, use the official AI Blueprint documentation and repository; the video is used as workflow context.

2. What AI Blueprint Actually Is

AI Blueprint is a file-backed, spec-driven AI coding workflow framework. It sits around your application and your coding agent. It is not a SaaS starter, UI kit, database framework or replacement for your application architecture. Its job is to make planning, implementation, verification, review and history durable and explicit.

Concept	Meaning
File-backed state	Important project decisions live in readable repository files rather than only inside a chat.
Spec-driven	A feature is specified before implementation and the developer reviews the spec.
One work item at a time	Each feature, bug fix or rollback has a bounded scope and acceptance criteria.
Evidence before completion	The agent must verify behavior instead of treating compilation as proof.
Human approval boundaries	Merge, push, deploy, publish and destructive actions remain approval-controlled.
Tool-independent project state	Codex, Claude Code, Copilot and OpenCode can share the same Blueprint state.

3. The Engineering Mindset

Role	You	AI agent
Product owner	Define users, goals, priorities and non-goals.	Clarify and organize requirements.
Architect	Approve stack, boundaries, data flow and trade-offs.	Inspect code and propose options.
Engineer	Review changes and understand important decisions.	Write and modify code.
Tester	Decide what must be demonstrably true.	Run tests and verification.
Reviewer	Judge whether risks are acceptable.	Find defects and inconsistencies.
Release owner	Approve merge/deployment/publication.	Prepare readiness configuration.

The key shift is from “AI, build my app” to “AI, execute this reviewed engineering task inside this controlled project.”

4. Before Installation

- Choose your application language/framework and architecture first.
- Create/scaffold the application.
- Initialize Git if the scaffolder did not.
- Confirm the application runs before adding Blueprint.
- Decide whether you will use Codex, Claude Code, or both.
- Keep credentials and production secrets outside prompts and planning files.

```
# Example
npx create-next-app@latest my-app
cd my-app
git init

# Install Blueprint
npx create-ai-blueprint@latest
```

Requirement: The current repository documentation lists Node.js 22+ plus an existing application and Git repository as requirements.

pnpm: If devEngines.packageManager enforces pnpm, use `pnpm dlx create-ai-blueprint@latest` instead of bypassing that requirement.

5. Adapter Selection

Tool	Skills	Typical invocation
Codex	.agents/skills/	\$feature, \$implement, \$check
Claude Code	.claude/skills/	/feature, /implement, /check
GitHub Copilot	AGENTS.md + .agents/skills/	Ask Copilot to run the matching skill
OpenCode	AGENTS.md + compatible shared skills	Ask OpenCode to run the matching skill

```
# Examples
npx create-ai-blueprint@latest -- --codex
npx create-ai-blueprint@latest -- --claude
npx create-ai-blueprint@latest -- --codex --opencode
npx create-ai-blueprint@latest -- --all
```

Shared state: All supported adapters use the same `blueprint/` project state, so switching agents does not require exporting the workflow history from chat.

6. First Project Setup

6.1 Onboard

Run onboarding inside the AI coding environment. It inspects the actual stack, commands, coding conventions, adapters, ignore rules, project configuration and existing verification/CI.

```
/onboard    # Claude Code
$onboard    # Codex
```

Do not skip this: Onboard is how the workflow learns the repository you are actually working in rather than an imagined repository.

6.2 Create the Two Planning Files You Own

File	Purpose
blueprint/project-plan.md	The what and why: problem, users, features, data, stack, constraints, UX, deployment and established non-requirements.
blueprint/build-plan.md	The ordered roadmap: one feature-sized outcome per line.

```
# blueprint/project-plan.md
# Project Plan

## Problem
What problem are we solving?

## Users
Who uses the system and what roles exist?

## MVP
What must work in V1?

## Data
What entities, relationships and sensitive data exist?

## Tech
What stack and external services are being used?

## UI/UX
Important screens, states and interaction rules.

## Security
Secrets, authentication, authorization and trust boundaries.

## Deployment
Where and how will the system run?

## Constraints
Known scale, latency, tenancy, compliance, offline requirements, etc.

## Non-goals
What are we deliberately NOT building?

# blueprint/build-plan.md
# Build Plan
```

- [] 1. Application shell - establish layout and navigation
- [] 2. Authentication - implement sign in and sign out
- [] 3. Core data - create and retrieve primary domain entities
- [] 4. Core workflow - implement the main user action
- [] 5. Error handling - handle expected failure states
- [] 6. Testing - add focused automated coverage
- [] 7. Deployment readiness - verify production configuration

Feature sizing: “Database” or “Frontend” is too broad. “Create and retrieve customer profiles” is a feature-sized outcome.

6.3 Optional Discovery

Use `/discovery` when requirements are still unclear. It asks focused questions, follows consequential answers and produces drafts of both planning files for your approval before writing them.

`/discovery`

Your authority: Discovery helps expose missing requirements; it does not transfer product decisions from you to the model.

6.4 Overview

`/overview`

Overview generates durable project context from your two owned plans. When product direction changes materially, update the owned plans and run Overview again instead of manually editing generated context.

7. The Core Workflow You Should Memorize

`/feature → /implement → /check → /audit current → /complete`

Stage	Command	Job	Human checkpoint
Investigate	<code>/explore</code>	Explore an idea/trade-off without changing files.	Decide whether it should be pursued.
Understand	<code>/brief</code>	Explain an existing planned item.	Resolve ambiguity.
Specify	<code>/feature</code>	Create the buildable feature specification.	Read and approve before implementation.
Build	<code>/implement</code>	Implement the approved specification.	Review scope and important decisions.
Prove	<code>/check</code>	Verify real behavior against acceptance criteria.	Judge whether evidence is sufficient.
Review	<code>/audit current</code>	Review implementation quality and risks.	Resolve serious findings.
Land	<code>/complete</code>	Run final gates and archive the work.	Approve landing/merge.

Important: The documentation presents Audit in the teaching path, but Audit/Check/manual-guide gates remain configurable; do not assume every project requires the same gates.

8. Complete Command Reference

Command	Purpose
/onboard	Tune a fresh Blueprint installation to the actual project.
/adopt	Bring Blueprint into an existing codebase with shipped behavior.
/discovery	Develop project and build plans through guided questioning.
/overview	Generate durable project context from the plans.
/explore	Investigate an idea or trade-off without writing files.
/brief	Explain a planned feature before specification.
/feature	Turn one planned item into a buildable spec.
/implement	Build the approved spec.
/check	Verify observable behavior against the current spec.
/check guide	Give human manual review steps; it does not itself prove success.
/audit	Review code and record findings.
/audit current	Review the current implementation delta.
/audit independent current	Request an independent fresh-context review of a checkpoint.
/complete	Run final gates, archive work and request approval to land it.
/status	Show progress, drift, blockers and suggested next action.
/doctor	Check Blueprint setup and workflow health.
/debug	Reproduce and isolate a failure without fixing it.
/fix	Spec a known bug or small unplanned change.
/tests	Set up unit testing.
/tests browser	Set up automated browser testing.
/ci	Create one documented Verify recipe and matching GitHub checks.
/prototype	Create throwaway visual prototypes.
/release	Prepare deployment configuration and readiness checks.
/rollback	Plan a history-preserving reversal.
/autopilot	Run one bounded feature/fix through configured gates.
/continuous	Process remaining planned features serially with local completion.

9. How to Execute One Feature Correctly

Step 1 — Understand the item

```
/status  
/brief 3
```

Use ``/brief`` when the next feature is large or unfamiliar. It is a pre-spec explanation, not implementation.

Step 2 — Generate the specification

```
/feature
```

Read ``blueprint/context/current-feature.md`` carefully. Check scope, acceptance criteria, data changes, dependencies, edge cases, security boundaries and testing expectations.

- Does the feature solve the intended problem?

- Are the acceptance criteria observable?
- Are edge cases explicit?
- Did the agent invent requirements?
- Did it add unnecessary infrastructure?
- Does the proposed approach fit the existing architecture?
- Is the feature small enough to review?

Golden rule: If the spec is wrong, stop and correct it. Do not implement first and hope to repair requirements later.

Step 3 — Implement

/implement

- Watch changed files.
- Watch for unrelated refactors.
- Watch for new dependencies.
- Watch for new services or infrastructure.
- Watch for silent removal of existing behavior.
- Ask the agent to stop when an architectural decision falls outside the approved scope.

Step 4 — Verify behavior

/check

Check is behavioral proof, not merely compilation.

Claim	Evidence that actually supports it
Parser/validator works	Focused tests including malformed and boundary inputs.
Responsive UI works	Production build plus browser observation/screenshot.
Form/navigation flow works	Real browser interaction and console/network evidence.
API works	Representative success, validation and failure requests.
Repository is healthy	The configured Verify recipe.

Step 5 — Audit

/audit current

Audit asks whether the implementation has important defects or risks, including correctness, security, performance, maintainability, testing and regressions. A clean Audit does not replace behavioral Check.

Step 6 — Complete

/complete

Completion performs the final configured gates, archives the work and requests approval before landing. Blueprint is designed not to silently merge, push, deploy or publish.

10. How to Write Better Plans

Area	Questions
------	-----------

Problem	What is wrong today? What outcome matters?
Users	Who uses it? What roles and permissions exist?
Workflows	What are the most important user/system flows?
Data	What entities, relationships and retention rules exist?
APIs	What external APIs exist? Auth? Limits? Failure behavior?
Architecture	What frontend/backend/database/services are needed now?
Security	What are the trust boundaries and secret-handling rules?
UX	Loading, empty, error, success, responsive and accessibility states?
Observability	What logs/metrics/audit records are necessary?
Testing	What must be unit-tested, browser-tested or manually verified?
Deployment	Environment variables, build/start commands, health checks, hosting?
Non-goals	What attractive features are explicitly postponed?

A particularly important Blueprint principle is proportional engineering: do not add abstractions, dependencies, services or specialized machinery merely because the system might eventually need them. Unknown scale is not evidence for enterprise architecture. At the same time, real trust and data-integrity boundaries still matter.

11. Testing and CI

Blueprint treats testing as explicit. It cannot know the correct runner for every technology stack, so testing is opt-in.

```
/tests
/tests browser
/ci
```

Layer	Purpose
Unit tests	Exercise deterministic application logic.
Browser tests	Exercise repeatable UI and user-flow behavior.
Verify	One project-owned recipe combining configured typecheck, tests and build.
GitHub Actions	Runs that exact Verify recipe on configured PR/default-branch events.
GitHub ruleset	Optional remote policy that can prevent merge until checks pass.

Current documentation describes Verify as typecheck → tests (when configured) → build. Existing CI is inspected and preserved rather than blindly replaced.

Avoid false confidence: An empty test suite should not be treated as proof. A build does not prove a click flow. A screenshot does not prove a malformed-input validator.

12. UI/UX Prototyping

```
/prototype
```

Use Prototype for throwaway visual exploration before production implementation. It is useful for layout, hierarchy and interaction direction. Do not allow prototype code to silently become your production architecture.

13. Existing Codebases

For a project that already has shipped behavior, use adoption rather than treating it as a fresh scaffold.

```
npx create-ai-blueprint@latest  
/adopt  
/overview
```

- Review the generated plans.
- Preserve working conventions unless there is a reason to change them.
- Use ``/status`` and ``/doctor`` to understand the current workflow state.
- Put new work through the same feature loop.

14. Debug vs Fix

Situation	Use	Why
You know something is broken but not why	<code>/debug</code>	Reproduce and isolate before editing.
You know the bug and desired correction	<code>/fix</code>	Create a bounded change specification.
The correction belongs to a planned feature	<code>/feature</code>	Keep it attached to product scope.

15. Autopilot and Continuous Mode

Autopilot

```
/autopilot
```

Autopilot is bounded automation for one explicit feature or fix. It still respects configured gates and stops before completion. It should be considered an optimization after you understand the manual workflow.

Continuous

```
/continuous
```

Continuous processes planned build-plan items serially, using local Git completion and stopping on decisions, unsafe state, failed checks or serious unresolved findings. It does not push.

Recommendation: Learn and use the manual loop first. Automate repetition only after you understand the failure modes.

16. Git as a Safety System

- Start from a known Git state.
- Keep features bounded.
- Inspect diffs frequently.
- Separate unrelated refactors.

- Use branches/workflow isolation for feature work.
- Use rollback for history-preserving reversal.
- Never approve a merge merely because the agent says “done”; approve because evidence supports the acceptance criteria.

```
git status
git diff
git log --oneline --decorate -10
```

17. Security Rules

- Never place production secrets in prompts or Blueprint plans.
- Keep `.env` and secret stores properly ignored.
- Never expose server credentials to frontend code.
- Treat authentication and authorization as separate concerns.
- Validate input at trust boundaries.
- Review every new dependency.
- Review AI-generated security claims against the actual implementation.
- Use independent review for sensitive or unusually broad work when available.

18. The Blueprint File Model

Path	Meaning	Ownership
blueprint/project-plan.md	Product/system direction	You own it
blueprint/build-plan.md	Ordered roadmap	You own it
blueprint/context/project-overview.md	Generated durable project context	Regenerate through Overview
blueprint/context/current-feature.md	Active feature/fix/rollback spec	Workflow-generated
blueprint/context/findings.md	Durable audit findings	Workflow-managed
blueprint/context/review.md	Independent review handoff/receipt	Workflow-managed
blueprint/history/	Archived work	Workflow-managed
blueprint/config.json	Shared workflow policy	User-owned configuration

Context management: The repository becomes durable memory. You can clear a chat and resume from project files instead of keeping one enormous conversation alive.

19. Anti-Patterns to Avoid

Bad practice	Why	Better approach
“Build my whole SaaS” in one prompt	Scope and context become unstable.	Plan and implement feature-sized slices.
Implement before reviewing spec	Wrong assumptions become expensive code.	Approve `/feature` first.
Green build = done	Compilation is not behavioral proof.	Use `/check` with direct evidence.
Fix everything while coding	Scope becomes unreviewable.	Keep unrelated work separate.
Add architecture for hypothetical scale	Creates unnecessary complexity.	Use the smallest reversible design that meets current needs.
Chat is the only memory	Decisions disappear across sessions.	Persist state in Blueprint files.
Automate before learning the workflow	You automate mistakes faster.	Learn manual loop first.
Trust the agent’s own success claim	Self-report is not independent	Run tests/checks/audit and review

	evidence.	evidence.
--	-----------	-----------

20. Your Daily Operating Procedure

1. Open the repository and run ``/status``.
2. Check Git state and unexplained changes.
3. Read the next unchecked build-plan item.
4. Use ``/brief`` if it is not immediately clear.
5. Run ``/feature``.
6. Read and challenge the generated specification.
7. Approve only when the spec matches your intention.
8. Run ``/implement``.
9. Inspect the diff and important implementation decisions.
10. Run ``/check``.
11. If evidence fails, fix the feature rather than declaring success.
12. Run ``/audit current`` when appropriate.
13. Resolve serious findings.
14. Run ``/complete``.
15. Review history and the next build-plan item.
16. Only then perform the relevant merge/push/deploy action with approval.

21. Reusable Project-Level Instruction

You are working inside a project managed with AI Blueprint.

Rules:

1. Treat `blueprint/project-plan.md` and `blueprint/build-plan.md` as user-owned product decisions.
2. Do not invent major product requirements.
3. Work on one scoped feature at a time.
4. Never implement an unapproved feature specification.
5. Keep changes inside the approved scope.
6. Prefer the smallest reversible solution that satisfies the current requirement.
7. Reuse existing architecture and dependencies before adding new ones.
8. If a decision materially changes architecture, data, security, cost or scope, stop and ask.
9. Prove behavior with direct evidence, not merely a successful build.
10. Never claim something is tested unless the relevant check actually ran.
11. Keep secrets out of code, plans and logs.
12. Explain important implementation decisions and uncertainties.
13. Never merge, push, deploy, publish or perform destructive actions without the workflow's approval boundary.

Use carefully: The installed Blueprint skills and repository instructions remain authoritative. This block expresses the operating philosophy; it is not a replacement for the official workflow files.

22. Exact Playbook for Your Upcoming Project

17. Describe the problem, users and desired outcome.
18. Define the MVP and write explicit non-goals.
19. Choose the stack based on actual requirements.
20. Scaffold the application and verify it runs.
21. Initialize Git.
22. Install AI Blueprint.
23. Run ``/onboard``.
24. Create ``project-plan.md``.
25. Create ``build-plan.md`` with feature-sized outcomes.
26. Run ``/overview``.
27. Optionally configure ``/tests`` and ``/ci``.
28. Use ``/brief`` for a complex first feature.
29. Run ``/feature``.
30. Review and approve the spec.
31. Run ``/implement``.
32. Run ``/check``.
33. Run ``/audit current`` where appropriate.
34. Run ``/complete``.
35. Repeat feature-by-feature.
36. Near release, use ``/release``.
37. Only after the manual workflow is reliable consider ``/autopilot`` or ``/continuous``.

23. Definition-of-Done Checklist

- Feature exists in the build plan.
- Specification is explicit and approved.
- Only approved scope was implemented.
- No unexplained unrelated files changed.
- Relevant automated tests exist where appropriate.
- Verify passes when configured.
- Real user/system behavior was checked.
- Error and edge cases were considered.
- Security boundaries were reviewed.
- Audit findings are resolved or consciously accepted.
- History is archived.
- Git state is understood.
- No merge/push/deploy/publish occurred without approval.

24. Quick Troubleshooting

Problem	First move
Agent seems to forget the project	Run <code>`/status`</code> ; inspect project overview; rerun <code>`/overview`</code> after material plan changes.

Workflow setup seems wrong	Run <code>`/doctor`</code> .
You do not know what to build next	Run <code>`/status`</code> and inspect <code>`build-plan.md`</code> .
You have an idea but are unsure whether to build it	Use <code>`/explore`</code> .
Feature is unclear	Use <code>`/brief`</code> .
Feature is too large	Split it into smaller feature-sized outcomes.
Unknown bug cause	Use <code>`/debug`</code> .
Known small bug	Use <code>`/fix`</code> .
Need manual verification instructions	Use <code>`/check guide`</code> .
Need stronger fresh review	Use <code>`/audit independent current`</code> .
Need CI	Use <code>`/ci`</code> .
Need browser automation	Use <code>`/tests browser`</code> .
Need to reverse completed work	Use <code>`/rollback`</code> .

25. Sources Used

- AI Blueprint official site — <https://ai-blueprint.dev/>
- AI Blueprint GitHub repository — <https://github.com/aibblueprintheq/ai-blueprint>
- Getting Started — <https://ai-blueprint.dev/docs/getting-started>
- Core Workflow — <https://ai-blueprint.dev/docs/core-workflow>
- Command Guide — <https://ai-blueprint.dev/docs/command-guide>
- Writing Your Plans — <https://ai-blueprint.dev/docs/writing-your-plans>
- Tool Adapters — <https://ai-blueprint.dev/docs/tool-adapters>
- Testing and CI — <https://ai-blueprint.dev/docs/testing>
- Supplied YouTube video — <https://youtu.be/L4g6GGLzAyo>

Research note: the YouTube page exposed limited machine-readable content in this environment, so the video's workflow claims were cross-checked against the supplied repository and current official documentation. Current command behavior in this guide follows the official documentation.

26. Final Principle

Do not use AI to remove engineering discipline. Use AI to make disciplined engineering faster.

If you consistently make the agent state what it intends to build, approve that scope, observe the implementation, demand behavioral evidence, review the code, and preserve the result in project files, you are no longer blindly vibe coding. You are operating an AI-assisted engineering workflow.

When you provide the actual project idea, the next step should be to create its `project-plan.md` and `build-plan.md`, then initialize the repository and execute the workflow one feature at a time.